

KnowledgeBase + Time Task Tracker

UPDATED SYSTEM ARCHITECTURE OVERVIEW

KnowledgeBase is a document and meeting intelligence platform that ingests content, supports semantic retrieval and grounded chat, and operationalizes meeting summaries through Time Task Tracker integration. The current cloud configuration uses Nomic for embeddings and **watsonx.ai** for LLM inference.

Platform workflow

1. Frontend

Users work through the React application for upload, search, chat, source management, and meeting workflows.

2. Backend

FastAPI receives requests and routes them into ingest, search, retrieval, summarization, and push workflows.

3. Knowledge layer

Embeddings, pgvector retrieval, and the active LLM provider support grounded response generation.

4. Meeting output

Meeting summaries can be pushed to Time Task Tracker while indexed content remains searchable.

5. Storage

PostgreSQL with pgvector stores chunked content, vectors, metadata, and meeting time records.

6. Time Task Tracker

Time Task Tracker stores meeting-based work logs and makes them available for downstream reporting and retrieval in chat.

Core pipelines

Pipeline	Description
Document ingest	Files uploaded through the UI or API are parsed, chunked, embedded, and stored in the documents table. This creates the reusable knowledge layer for search and chat.
Semantic search	User queries are embedded and matched against stored vectors in pgvector. Results are returned directly with source context and relevance ordering.

RAG chat	Relevant chunks are assembled into grounded context and sent through the active LLM provider. Responses return with supporting source references.
Meeting summarization	Meeting transcripts move through the same ingestion pipeline, then generate structured summaries that can be pushed into Time Task Tracker.

Provider configuration

Current cloud path

Nomic generates embeddings and watsonx.ai handles LLM inference through the IBM IAM token exchange and the **/ml/v1/text/chat** endpoint.

Component	watsonx.ai mode
Embed provider	EMBED_PROVIDER=nomic
Embed model	nomic-embed-text-v1.5
LLM provider	LLM_PROVIDER=watsonx
LLM model	meta-llama/llama-3-3-70b-instruct
Transport	IBM IAM token exchange and watsonx chat API

Time Task Tracker capabilities

Operational features

Time Task Tracker is more than a write target for meeting summaries. It stores structured time-entry data that can be queried back into the RAG workflow for historical lookup and reporting.

Capability	How it is used
Meeting time logging	Stores each pushed meeting record with project code, task type, duration, date, description, and meeting title.
Structured metadata	Captures billable flag, confidence, status, organizer, and attendees for each meeting entry.

Project-based reporting	Supports grouping and filtering by project code and task type for workload and effort tracking.
Billable and status filtering	Allows retrieval of billable entries and workflow state through the query layer.
Historical meeting retrieval	Feeds prior meeting records back into chat when users ask for recent or past meeting history.

Data model and key files

Data model

The **documents** table stores chunked source content, embeddings, and metadata. The **time_entries** table stores meeting records with project, duration, status, attendees, and related operational fields after successful summarization and parsing.

File	Responsibility
backend/api.py	REST endpoints for upload, search, chat, sources, and meeting ingest
backend/kb/config.py	Environment-driven configuration, including watsonx.ai values
backend/kb/llm.py	Provider abstraction for Ollama, OpenAI-compatible APIs, and watsonx.ai
backend/kb/chat.py	RAG orchestration and grounded answer generation
backend/kb/ingest.py	Document and transcript ingestion pipeline
backend/kb/pusher.py	Time Task Tracker integration for meeting summaries and time-entry insertion
backend/kb/ttt.py	Time Task Tracker query layer used to pull historical entries into chat context
frontend/src/components/MeetingUpload.jsx	Meeting ingestion UI and result display
render.yaml	Backend deployment configuration